Exercise 1: dbt Setup and Project Structure

Objective

Install dbt with the DuckDB adapter, create your first dbt project, and understand the project structure and connection configuration.

About DuckDB

For this first class we use **DuckDB** as our database. DuckDB is:

- **Embedded**: runs in-process, like SQLite — no server needed
- **Analytical**: columnar storage optimized for analytics queries
- **SQL-complete**: supports CTEs, window functions, JOINs, aggregations
- **dbt-compatible**: the `dbt-duckdb` adapter is officially supported

In the next class we will replace DuckDB with **Snowflake**. The same dbt code runs on both — only the connection profile changes.

Prerequisites

- Python 3.9+ installed (`python --version` to verify)
- Terminal access

Step 1: Create a virtual environment and install dbt

```
mkdir ~/dbt-class && cd ~/dbt-class
python -m venv .venv
source .venv/bin/activate
pip install dbt-duckdb
```

On Windows use `.venv\Scripts\activate` instead.

Verify the installation:

```
dbt --version
```

Expected output:

```
Core:
  - installed: 1.9.x
  ...
Plugins:
  - duckdb: 1.9.x - Up to date!
```

Step 2: Create a dbt project

```
dbt init airbnb
```

dbt will ask which database adapter to use. Enter the number corresponding to `duckdb`.

Move into the project folder:

```
cd airbnb
```

Step 3: Configure the connection

dbt stores connection settings in `~/.dbt/profiles.yml`. This file lives **outside** the project on purpose — it may contain credentials that should never be committed to Git.

Open `~/.dbt/profiles.yml` in your editor and replace its entire content with:

```yaml
airbnb:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: "airbnb.duckdb"
      threads: 4
```

This tells dbt to create a file called `airbnb.duckdb` in the working directory whenever it runs commands.

Step 4: Explore the project structure

```
airbnb/
├── dbt_project.yml        # Project configuration: name, paths, default settings
├── profiles.yml           # (NOT here — credentials live in ~/.dbt/profiles.yml)
├── models/
│   └── example/
│       ├── my_first_model.sql     # Example view model
│       ├── my_second_model.sql    # Example table model (depends on first)
│       └── schema.yml             # Tests and documentation
├── seeds/                 # CSV files that dbt loads as database tables
├── snapshots/             # SCD Type 2 history tracking
├── macros/                # Reusable Jinja2 functions
├── tests/                 # Custom SQL tests
└── analyses/              # Ad-hoc SQL files (not materialized)
```

Open `dbt_project.yml` and read through it. Key fields:

Field	Purpose
`name`	Project name, must match the profile name in `profiles.yml`
`profile`	Which profile in `~/.dbt/profiles.yml` to use

`model-paths`	Where dbt looks for `.sql` model files
`models`	Default materialization config per folder

Step 5: Verify the connection

```
dbt debug
```

This checks every component: your profile file, your project file, and the database connection. All checks should pass with green checkmarks.

```
All checks passed!
```

If you see a connection error, verify the path in `~/.dbt/profiles.yml` and that you are running `dbt debug` from inside the `airbnb/` directory.

Step 6: Run the example models

```
dbt run
```

dbt compiles each `.sql` file and executes it against DuckDB. You should see:

```
1 of 2 START sql view model dev.my_first_model ........... [RUN]
1 of 2 OK created sql view model dev.my_first_model ..... [OK in 0.07s]
2 of 2 START sql table model dev.my_second_model ........ [RUN]
2 of 2 OK created sql table model dev.my_second_model ... [OK in 0.05s]
```

Step 7: Inspect what was created

A file called `airbnb.duckdb` now exists in your directory. Inspect it with Python:

```python
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("SHOW ALL TABLES").df())
conn.close()
```

You will see two objects: `my_first_model` (a view) and `my_second_model` (a table).

Step 8: Read the compiled SQL

dbt compiles your Jinja templates into plain SQL before executing them. Look at what was actually sent to DuckDB:

```
cat target/compiled/airbnb/models/example/my_first_model.sql
```

Now open the source file:

```
cat models/example/my_second_model.sql
```

Notice the `{{ ref('my_first_model') }}` call. dbt resolves this to the actual table/view name at compile time, and also uses it to build the **dependency graph** (the DAG).

Step 9: Run the built-in tests

```
dbt test
```

The `schema.yml` file in `models/example/` defines two generic tests: `unique` and `not_null` on the `id` column of `my_second_model`.

Key Concepts

Concept	What it means
Profile	Connection settings (adapter, host, database). Lives outside the project.
Project	All your SQL models, config, and macros. Committed to Git.
`dbt run`	Compiles Jinja + executes SQL against the database
`dbt test`	Runs data quality assertions defined in YAML
`dbt debug`	Validates config and tests the database connection
`ref()`	References another model and builds the DAG automatically

---

Questions

1. Why does `profiles.yml` live in `~/.dbt/` and not inside the project folder?
2. What is the difference between `target/compiled/` and `target/run/`?
3. If `my_second_model` depends on `my_first_model`, what happens when you run `dbt run --select my_second_model` without running `my_first_model` first?